

Notas de construcción

Cómo pensé y armé un portal de reporte ejecutivo para Palvi — desde leer la consigna hasta el último ajuste de UI.

01

El punto de partida

Lo primero fue leer el brief y el **metrics.json** con calma. La forma de los datos importa más que cualquier feature: 4 datasets (A, B, C, D) con la misma estructura — 11 métricas, 365 días — pero cada uno cuenta una historia distinta. El usuario es el Jefe de Ventas, abre la página antes de su primera reunión, tiene 5 minutos, y necesita salir sabiendo dónde poner foco hoy. Eso me dejó claras dos cosas: el dashboard tenía que **responder distinto a cada dataset**, y la unidad de información no son los números sueltos sino las **señales accionables** que el equipo puede atender con su gente.

02

Por qué un portal con login

Aunque el brief no lo pedía, decidí montarlo como un portal con autenticación. La razón es de uso: este reporte es para una sola persona (el responsable comercial) y la información que ve es sensible — pipeline, deals perdidos, capacidad del equipo. Un dashboard sin auth público de nuevo en la mañana rompe ese encuadre. Login simple, sin registro (sistema interno), un admin sembrado de fábrica. Esa decisión la defendí en el README: el brief evalúa criterio, y el criterio acá decía agregar una capa delgada de protección antes de cualquier feature.

03

La interfaz, antes que el código

Le pedí a **Claude Design** un primer prototipo de la pantalla principal. Le pasé contexto del dominio y de cómo iba a usarse, y volví con un Reporte Matutino diseñado: header con saludo y rango de fechas, sección de Foco de hoy con tarjetas priorizadas, embudo de conversión con tasa entre etapas, grilla de KPIs ordenada por urgencia, y un bloque de soporte. Tipografía Geist + Geist Mono, paleta warm-neutral, énfasis en el dato y no en la decoración.

El bundle exportado por Claude Design venía con CSS, JSX y datos de ejemplo. Mi trabajo fue traducir ese diseño a producción — copiar el CSS literal, repensar los componentes en TypeScript con tipos fuertes, y conectar todo con datos reales. La consigna fue clara: **match visual pixel-perfect**, no copiar la estructura interna del prototipo si no calza con el código real.

04

El stack y por qué

Frontend: React + TypeScript + Vite. React y TS los pedía el brief. Vite por velocidad de iteración y porque encaja con la naturaleza ligera del proyecto — no necesitaba SSR ni router pesado.

Backend: Python con FastAPI + SQLAlchemy async + Postgres. Acá el brief dejaba libertad. Elegí Python porque va de la mano con datos: si mañana la ingesta deja de ser un JSON estático y se convierte en CSVs nocturnos, validación de pipelines, o reglas más complejas, Python tiene la biblioteca y la cultura para hacerlo sin dolor. FastAPI levanta una API tipada en minutos, SQLAlchemy 2 async se integra limpio con asyncpg, Postgres porque ENUMs nativos y composición de queries.

Docker Compose para levantar los tres servicios con un solo comando — nadie debería tener que instalar Postgres en su máquina para correr esto.

Sé que para 668 KB de JSON estático un backend Python es over-engineering. El brief evaluaba decisiones; preferí defender una arquitectura que escale a cosas reales — ingesta automatizada, auth, evaluación server-side de reglas — antes que entregar una SPA que lee un fixture.

05

El flujo de trabajo: SDD con Claude Code

Acá está la parte que más cambió mi forma de construir. Mi setup de Claude Code tiene cargado un sistema de **Spec-Driven Development** (SDD): cada cambio no trivial pasa por un ciclo cerrado antes de tocar código.

Las fases del ciclo: explore → propose → spec → design → tasks → apply → verify → archive. Cada fase la corre un sub-agente especializado, con un contrato claro de qué lee y qué escribe. Yo orquesto la cadena pero cada artefacto queda persistido en memoria (engram), así que si la sesión se compacta o vuelvo otro día, el estado del cambio está intacto.

Por qué importa esto:

- **No salto a programar.** Antes de tocar un archivo, hay una propuesta, un spec con escenarios de aceptación y un design con decisiones explícitas. Esto fuerza pensar el problema antes que la solución.
- **Decisiones quedan documentadas.** Cada ADR (architectural decision record) vive con el cambio. Cuando un sub-agente desviaba del spec con justificación, quedaba registrado.
- **Verificación al final.** Un agente independiente revisa el código contra el spec — separa warnings de bugs reales y propone arreglos antes del archive.
- **Recovery.** Si en una iteración me equivocaba en la lectura del requerimiento (me pasó), tenía el ciclo entero como historial para entender dónde se torció y proponer un nuevo cambio que rectifique.

El proyecto cerró 5 ciclos SDD completos: **multi-dataset-ingestion**, **auth-login**, **alarm-rules-engine**, **rules-manager**, **metrics-date-filter**. Cada uno con su set de artefactos en engram, su verify report, su archive.

06

Las iteraciones, en orden

1. Multi-dataset ingestion

El primer scaffold seedeaba un dataset sintético — error de arranque, descubierto al leer mejor el brief. Reemplacé el seed por la carga del *metrics.json* real con un módulo *ingest.py* idempotente y un flag *FORCE_RESEED*. Schema: tabla *datasets* + FK no-nulo en *metric_values*. Frontend: switcher A/B/C/D en el header con estado en URL (*?dataset=A*) y cache local *Map<id, Dataset>*.

2. Auth login

JWT HS256 + bcrypt, sin registro. Token en localStorage (XSS aceptado por ser herramienta interna). State-gate en *main.tsx* sin librería de routing — un componente decide entre *LoginPage* y *App*. Default *admin@palvi.local* / *palvi* sembrado al boot. Todos los routers existentes pasaron a requerir token con una sola línea (*dependencies=[Depends(get_current_user)]* a nivel router).

3. Alarm rules engine — el corazón

La primera versión de las alarmas era texto libre. La rechacé después de hablarlo con vos: una alarma tiene que ser una **regla evaluable**, no una nota pegada en pantalla. Cambio profundo: *Alarm* redefinida como *nombre + métrica + operador + umbral + severidad*. El operador es un ENUM Postgres con 6 valores (lt, lte, gt, gte, eq, neq), inválidos imposibles a nivel DB. Un módulo puro *rules.py* con *evaluate_rule()*. El endpoint *GET /api/alarms?dataset=A* hace UN solo query *DISTINCT ON* para traer el último valor de cada métrica del dataset y devuelve cada regla con *triggered* y *current_value*. **Foco de hoy** solo pinta las disparadas.

4. Rules manager

Detectaste un gap: una regla que no dispara desaparece. No se puede listar, editar ni borrar. Agregué una vista de mantenedor (*?view=rules*) con CRUD inline sobre todas las reglas — disparadas o no — y badge "Activa hoy / Inactiva". Toggle Reporte/Reglas en el header. Backend sin cambios.

5. Metrics date filter

Hasta acá las ventanas eran fijas (últimos 7d vs anteriores 7d). Agregué filtro de rango libre con *?from=&to=*, presets 7d/30d/90d, y barra sticky bajo el header. Implementación: helpers *sliceByRange/aggregateSlice/periodDeltaSlice/buildFunnelFromSlices* en *metricUtils.ts* (additive, sin romper la API legacy). Las reglas evalúan contra el último día del rango — el endpoint *GET /api/alarms* acepta *?to=* opcional. Bonus: botón **Exportar CSV**, presets en negro cuando matchean, footer con narrativa del dataset removido (no agrega valor real).

07

La pieza central: alertas como reglas personalizables

Si tengo que defender una sola decisión del proyecto, es esta. Las métricas son siempre las mismas (11 keys, estructura constante), pero **qué importa** depende de la persona, del momento del año, del estado de la operación. Que la herramienta venga con alertas hardcodeadas "Response time alto" o "Stale deals fuera de control" es traicionar al usuario: lo

obliga a aceptar mi lectura del negocio.

La solución son **reglas que el responsable crea**: elige una métrica, un operador, un umbral, una severidad, le pone un nombre. La regla evalúa todos los días contra el último valor del rango elegido. Cuando dispara, la ve en Foco de hoy con el valor actual al lado. Cuando no dispara, sigue viva en el mantenedor para editarla o borrarla.

Esto convierte la herramienta en algo configurable. El día que la operación cambie — campaña nueva, objetivo distinto, métrica que importa diferente — la persona ajusta sus reglas y el reporte sigue siendo útil. La estructura de datos se respeta; la interpretación es del usuario.

08

Lo que dejé fuera (a propósito)

- **Tests automatizados.** Lo natural: unitarios para *metricUtils.ts* (sentiment, slicing, null-handling) y un par de e2e para los flujos del switcher y filtro. Es la lógica que más fácil rompe en silencio.
- **Reglas combinadas.** Hoy una regla compara un valor contra un umbral. Lo natural siguiente: *AND/OR*, deltas ("subió 50% en una semana"), tendencias ("3 días seguidos peor"). Requiere un mini DSL o un editor visual; preferí foco antes que ambición.
- **Reglas scoped por dataset.** Hoy son globales — el mismo umbral aplica a A/B/C/D. Si cada dataset necesita umbrales distintos, basta un FK opcional *dataset_id* en *alarms*.
- **Ingesta real.** Hoy se lee el JSON al boot. La iteración natural es un endpoint *POST /api/metrics/import* que acepte CSV/JSON + un cron que tire del CRM cada noche.
- **Drag & drop para reordenar reglas.** La columna *position* ya existe en la DB; falta el control de UI.

Repo: estructura monorepo con *frontend/* (React+TS+Vite), *backend/* (FastAPI+SQLAlchemy), *insumos/* (metrics.json + brief), *docker-compose.yml*. Levantar con *docker compose up --build*. Login: *admin@palvi.local* / *palvi*. Datasets se cargan al primer boot.